Playwright API Testing: Week 1 Notes (Review & Interview Prep)

Oct 25, 2026 · @Kiran

Setup and config

Week 1 goal: send GET, POST, PUT, PATCH and DELETE requests with Playwright's `request` fixture and assert on status, headers and JSON bodies. The practice API is JSONPlaceholder, which returns realistic responses but does not save any writes.

```
mkdir playwright-api-practice && cd playwright-api-practice
npm init playwright@latest
```

Choose TypeScript, keep the `tests` folder, and skip the GitHub Actions workflow for now. Then set `baseURL` in `playwright.config.ts` so tests can use short paths like `/posts/1`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  reporter: [['list'], ['html', { open: 'never' }]],
  use: {
    baseURL: 'https://jsonplaceholder.typicode.com',
    extraHTTPHeaders: { Accept: 'application/json' },
  },
});
```

Run commands:

Command	What it does
`npx playwright test`	Runs all tests
`npx playwright test posts.get.spec.ts`	Runs one file
`npx playwright test -g "users"`	Runs tests whose title matches
`npx playwright test --workers=1`	Runs serially, useful for debugging

Command	What it does
`npx playwright show-report`	Opens the HTML report

Core concepts

Four response calls cover most API tests, and each request option has one specific job.

Response methods

Call	Returns
`response.status()`	The numeric status code, e.g. 200
`response.ok()`	`true` for any 2xx status (it is a method, so it needs `()`)
`response.headers()`	Object of response headers, keys in lowercase
`await response.json()`	The parsed body
`await response.text()`	The raw body, useful for debugging

Request options

Option	Use
`params`	Query string, e.g. `{ params: { userId: 1 } }` becomes `?userId=1`
`data`	JSON request body for POST, PUT and PATCH
`form`	URL-encoded form body
`multipart`	File uploads
`headers`	Per-request headers, which override the config headers
`failOnStatusCode: true`	Throws on any 4xx or 5xx. Use it for setup calls that must succeed, and leave it off for negative tests

Assertion matchers

Matcher	Checks	Use for
`toBe`	Strict identity	Numbers and strings. `toBe({})` fails because two objects are different references
`toEqual`	Full structure	Exact objects or arrays. Fails if there are extra fields
`toMatchObject`	Only the listed fields	API bodies, because servers add fields like `id`
`toBeTruthy` / `toBeFalsy`	Truthiness	Booleans such as `response.ok()`
`await expect(response).toBeOK()`	Any 2xx status	Success checks where the exact code does not matter

Use `await` only on promises: `request.get`, `response.json()` and async matchers like `toBeOK()`. A value already in memory, such as `post.userId`, needs no `await`.

Exercises and corrected solutions

These are the Week 1 exercises in their corrected form. Rewrite each from memory before an interview, then break it once on purpose to confirm it can fail.

1. GET /users/1 returns user details

```
import { test, expect } from '@playwright/test';

test('GET /users/1 returns user details', async ({ request }) => {
  const response = await request.get('/users/1');
  expect(response.status()).toBe(200);

  const user = await response.json();
  expect(user.name).toBeTruthy();
  expect(user.email).toContain('@');
  expect(user.address.city).toBeTruthy();
});
```

2. GET /posts filtered by userId returns only that user's posts

```
interface Post { userId: number; id: number; title: string; body: string }

test("GET /posts filtered by userId returns only that user's posts", async ({
request }) => {
  const response = await request.get('/posts', { params: { userId: 1 } });
  expect(response.status()).toBe(200);

  const posts: Post[] = await response.json();
  expect(posts.length).toBeGreaterThan(0);

  for (const post of posts) {
    expect(post.userId, `post ${post.id} has the wrong userId`).toBe(1);
  }
});
```

User 1 has exactly 10 posts (ids 1 to 10), so you can also assert `toHaveLength(10)`.

3. Negative test: missing resource vs empty search

```
test('GET /posts/9999 returns 404 for a missing post', async ({ request }) =>
{
  const response = await request.get('/posts/9999');
  expect(response.status(), 'missing post should be 404').toBe(404);
  expect(await response.json()).toEqual({});
});

test('GET /posts?id=9999 returns an empty list, not an error', async ({
request }) => {
  const response = await request.get('/posts', { params: { id: 9999 } });
  expect(response.status()).toBe(200);
  expect(await response.json()).toEqual([]);
});
```

Data-driven version, which gives one report line per ID:

```
for (const id of [9999, 0, -1, 100000]) {
  test(`GET /posts/${id} returns 404`, async ({ request }) => {
    const response = await request.get(`/posts/${id}`);
    expect(response.status()).toBe(404);
  });
}
```

4. POST /comments creates a comment

```
test('POST /comments creates a comment', async ({ request }) => {
  const payload = {
    postId: 1,
    name: 'Kiran Test',
    email: 'kiran@example.com',
    body: 'Practicing Playwright API testing',
  };

  const response = await request.post('/comments', { data: payload });
  const body = await response.json();

  expect(response.status(), 'POST should return 201 Created').toBe(201);
  expect(response.headers()['content-type']).toContain('application/json');
  expect(body).toMatchObject(payload);
  expect(typeof body.id).toBe('number');
  expect(body.id).toBeGreaterThan(100);
});
```

JSONPlaceholder returned `id: 501` for this POST because it has 500 comments, and a later GET of `/comments/501` returns 404 because the write is faked. A real API would not let you assert a specific ID, only that it exists and has the right type.

Mistakes and lessons

Every bug below happened in practice, and each one is a common real-world and interview trap.

Mistake	What happened	Fix
`expect(response.ok).toBeTruthy()`	`ok` is a method, so the function itself is checked and it is always truthy. The assertion can never fail, even on a 500	Call it: `response.ok()`

Mistake	What happened	Fix
`expect(response.ok).toBeFalsy()`	Same bug in reverse: it failed with `Received: [Function ok]` even though the intent was right	`response.ok()`
`let posts: [] = ...`	`[]` is a zero-element tuple type, so `posts[i].userId` is a type error	Define an interface and use `Post[]`
`await posts[i].userId`	Nothing to await, because the value is already in memory	Remove the `await`
Looping over a response without checking status or length	If the call fails or returns `[]`, the loop never runs and the test passes with zero assertions	Assert the status first, then `length > 0`, then loop
`get('/posts', { params: { id: 9999 } })` expecting a 404	That is a filter on the list, so the server answers 200 with `[]`	Use the path form: `get('/posts/9999')`
`toBe(200)` and `toBeOK()` together	Redundant: an exact 200 already proves a 2xx	Use `status()` for an exact code, or `toBeOK()` for any 2xx, not both
`failOnStatusCode: true` in a 404 test	Playwright throws on the request line, before the 404 assertion runs	Leave it off for negative tests
Calling `response.json()` several times	Works, but re-parses each time	Store it once: `const body = await response.json()`

Lessons to carry forward

1. When a test passes on the first run, break it once on purpose (wrong URL, wrong expected value) to prove it can fail.
2. A filter that matches nothing and a resource that does not exist are different cases: 200 with an empty list versus 404.
3. If a filtered list contains rows that should not be there, suspect that the filter never reached the server. Log `response.url()` and the array length to check.
4. Add a custom message to assertions inside loops, so the failure names the item. For example `expect(post.userId, 'post 11 has the wrong userId').toBe(1)` shows as the first line of the error in the terminal and the HTML report.

Tips cheat sheet

Quick reminders to scan before writing or reviewing an API test.

Assertions

- Use `toMatchObject` for API bodies, since servers add fields like `id` and `createdAt`.
- Plain `expect(value)` is synchronous. Use `await expect(response)` for async matchers like `toBeOK()`, and never forget the `await`.
- `expect.soft(...)` records a failure and keeps going, so one run shows every broken field.
- Add a custom message as the second argument of `expect(value, 'message')`. It appears as the first line of the error.

Reading responses

- Check the `content-type` header before calling `.json()`. An HTML error page makes `.json()` throw a confusing parse error.
- Use `response.text()` when debugging, because it shows what actually came back.
- `response.url()` and `response.statusText()` are useful in logs when a test fails in CI.

Request options

- Prefer `params: { userId: 1 }` over hand-built `'/posts?userId=1'`, because Playwright handles encoding. The key must be spelled exactly `params`.
- `data` sends JSON, `form` sends URL-encoded form data, `multipart` sends files. The wrong type is a common cause of unexpected 400 errors.

- Per-request `headers` override the config headers, e.g. `{ headers: { Authorization: 'Bearer abc' } }`.
- Add the body to a failure message with `expect(response.status(), await response.text()).toBe(200)`.

Test habits

- One behavior per test, named by behavior, so a failing title tells you what broke.
- Each test creates its own data, because tests can run in parallel and in any order.
- Never hardcode generated values such as tokens or IDs. Extract them from responses.
- Assert types for generated data: `typeof body.id` should be `'number'`.
- Remove `console.log` once the test works. Assertions are what make a test valuable.

Interview Q&A

Short answers you can say out loud, based on what Week 1 covered.

What is the difference between 200, 201 and 204? 200 OK means the request succeeded and there is a body (typical for GET, PUT, PATCH). 201 Created means a new resource was created (typical for POST). 204 No Content means success with no body (typical for DELETE). Assert the exact code instead of just checking for 2xx, so a wrong-but-successful code is caught.

Which negative scenarios would you test for an API? 404 for a nonexistent ID, 400 for missing or invalid fields, 401 for missing or wrong credentials, and 403 for valid credentials without permission. Also check the error message and body, not just the status, and confirm no sensitive data or token is returned.

What is the difference between a 404 and an empty result? Requesting a specific resource that does not exist (`/posts/9999`) gives 404. Filtering a collection that matches nothing (`/posts?id=9999`) gives 200 with an empty list, because the search itself worked.

When do you use toBe, toEqual and toMatchObject? `toBe` is strict identity, for numbers and strings. `toEqual` compares the whole structure and fails on extra fields. `toMatchObject` checks only the fields you list, which suits API responses where the server adds fields like `id`.

How do you send query parameters and a request body in Playwright? Query parameters go in `params`, and the JSON body goes in `data`. Mixing them up is a common mistake: `data` on a GET does not filter anything.

Does Playwright throw on a 404 or 500 response? No, not by default. The response is returned and you assert on it. Setting `failOnStatusCode: true` makes it throw, which suits setup steps that must succeed but breaks negative tests.

How do you make sure a test can actually fail? Break it once on purpose: change the URL or the expected value and confirm it fails. A missing `()` on `response.ok` made an assertion that could never fail.

How do you stop a test from silently passing on an empty response? Assert the status first, then that the array is not empty, and only then loop over the items.

How do you handle generated values like IDs and tokens? Extract them from the response and reuse them. Do not hardcode them, and assert their type (`typeof body.id === 'number'`) instead of an exact value.

Why is JSONPlaceholder's POST not enough for a full CRUD test? It fakes writes: it returns a realistic response but saves nothing, so a follow-up GET of the new ID returns 404. A real create, read, update, delete cycle needs an API that persists data, such as Restful-Booker in Week 2.

Python equivalent and self-check

The concepts are identical in Python (pytest-playwright). Only the syntax differs, and the sync API means no `await` anywhere.

Concept	JS/TS	Python
Request context	`request` fixture	`playwright.request.new_context(base_url=...)`
GET / POST	`await request.get(url)`	`api.get(url)` / `api.post(url, data={...})`
JSON body	`await response.json()`	`response.json()`
Any 2xx check	`expect(response).toBeOK()`	`expect(response).to_be_ok()`
Exact status	`response.status()`	`response.status`
Setup and teardown	`beforeAll`, `afterAll`	pytest fixtures with `yield`
Parametrize	loops or arrays	`@pytest.mark.parametrize`

Concept	JS/TS	Python
Typing	interfaces, `zod`	Pydantic models or `TypedDict`

A good habit is to solve each exercise in TS first, then redo it in Python.

Self-check before starting Week 2

- [ ] I can write a GET test from scratch with status, header and body assertions
- [ ] I know why `response.ok` without `()` never fails
- [ ] I can explain `params` vs `data` and when to use each
- [ ] I can explain 404 vs an empty list, and test both
- [ ] I know when to use `toBe`, `toEqual` and `toMatchObject`
- [ ] I can explain what `failOnStatusCode: true` does and when to use it
- [ ] I can write a POST test that checks 201, the echoed body and a generated `id` type
- [ ] I break every new test once on purpose to prove it can fail
- [ ] I have redone at least two exercises in Python

Next: Week 2. Switch to Restful-Booker (a real persisting API) for token authentication, a full create, read, update, delete cycle, negative auth tests, and a second project in `playwright.config.ts` for the new base URL.